

APACHE DORIS 3.0.8

学习笔记

第四章：查询执行与优化

从一条 SQL 的完整旅程出发，串起 Nereids、统计信息、MPP、Pipeline、向量化、数据裁剪、Join、聚合、TopN、Runtime Filter 与慢查询诊断。

配套答案：notes-04-a

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

本章学习目标	4
先记住四条原则	4
4.1 一条 SQL 的完整旅程	5
四个计划层次	5
FE 与 BE 的职责	5
为什么局部计算很重要	5
三个执行维度	6
4.2 Nereids：从 SQL 到物理计划	6
RBO：先消除确定性的浪费	6
CBO：在多个合理方案中比较成本	6
Memo 与等价计划	7
RBO 与 CBO 对照	7
调优顺序	7
4.3 统计信息与基数估算	7
两个容易混淆的基数	7
主要统计指标	8
选择率的简化心算	8
误差如何传播	8
收集与检查	8
4.4 PlanFragment、Exchange 与 MPP	8
层级关系	9
为什么在 Exchange 处切 Fragment	9
一条聚合查询的 Fragment DAG	9
为什么 MPP 快但不线性	9
Doris 3.0 两种部署形态	9
4.5 Pipeline 与向量化	10
为什么不让任务永久绑定线程	10
阻塞算子如何切 Pipeline	10
背压与 Local Exchange	10
向量化为什么快	10
不要盲调 batch_size	11
4.6 Scan 与多层数据裁剪	11
分区裁剪与 Tablet 裁剪	11
让条件保持可推导	11
列裁剪与 SELECT *	12
ZoneMap 的确定性跳过	12
EXPLAIN 扫描检查	12

4.7 Hash Join 与四种分布策略	12
Build 与 Probe	12
Broadcast	13
Partition Shuffle	13
Bucket Shuffle 与 Colocate	13
四种策略总表	14
Join 顺序与结果膨胀	14
Hint 只做最后验证	14
4.8 聚合、Sort、TopN 与 Runtime Filter	14
局部聚合与全局聚合	14
Sort 与 TopN	15
Join Runtime Filter	15
Runtime Filter 的边界	15
TopN Runtime Filter	15
4.9 数据倾斜、并行度与内存	15
提高并行度的收益与代价	16
先判断瓶颈类型	16
五类倾斜	16
Profile 识别	16
查询内存构成	17
内存超限排查	17
Workload Group	17
4.10 EXPLAIN 与 Profile 慢查询诊断	17
工具分工	17
生成 Profile	17
MergedProfile 与明细 Profile	18
通用指标	18
从 Scan 向上读	18
标准诊断流程	18
教学案例：三层根因	18
为什么找第一处异常	19
第四章复习题	19
自测方法	20
术语速查	20
参考资料	20

本章学习目标

本章的目标不是记住一组调参命令，而是建立一套从 SQL 到真实执行的因果模型：优化器如何估算、物理计划如何分布、BE 如何调度、数据为何能被提前裁掉，以及怎样用证据定位第一处异常。

层次	核心问题	关键对象
规划	SQL 应该选择哪条执行路线？	Nereids、RBO、CBO、Statistics
分布式	计算如何拆到多个 BE？	Fragment、Instance、Exchange
单机执行	有限线程如何服务多个任务？	Pipeline、Task、Dependency、Block
减少工作	哪些数据可以根本不读？	Partition、Tablet、Column、ZoneMap
复杂算子	Join、聚合和排序如何控制数据量？	Hash、Shuffle、TopN、Runtime Filter
诊断	计划为何没有按预期运行？	EXPLAIN、Profile、max/avg/min

第四章主线：从声明式 SQL 到可验证的执行行为



先记住四条原则

- 少读通常比多开线程更有效；少传通常比网络堆带宽更有效。
- 估算影响计划，实际数据决定最终运行表现。
- 并行查询的阶段耗时通常由最慢 Instance 决定。
- 最慢算子可能只是受害者，要沿数据流寻找第一处异常。

4.1 一条 SQL 的完整旅程

SQL 只描述‘要什么’，优化器与执行引擎负责决定‘怎样做’。性能差异来自不同 SQL 被翻译成了不同的物理路线。

```
SELECT c.region, SUM(o.amount) AS revenue
FROM fact_order o
JOIN dim_customer c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2026-08-01'
      AND o.order_date < '2026-09-01'
      AND o.status = 'paid'
GROUP BY c.region
ORDER BY revenue DESC LIMIT 10;
```

FE 规划阶段



四个计划层次

层次	表达内容	暂时不关心
AST	SQL 的语法结构	数据如何分布
逻辑计划	Scan、Filter、Join、Aggregate 等关系运算	具体 BE 和 Shuffle
物理计划	Join算法、分布、聚合阶段、Exchange	运行时某个 Task 的实际耗时
执行实例	Fragment 在具体 BE 上的 Instance 与 Task	SQL 表面写法

FE 与 BE 的职责

- FE 是控制面：解析、元数据、权限、优化、生成 Fragment、协调执行。
- BE 是数据面：扫描 Tablet、执行 Join/Aggregate/Sort、交换 Block、产生结果。
- 数据密集型最终聚合通常在根 Fragment 所在 BE 完成；FE 协调并通过协议返回结果，而不是亲自扫描所有行。

为什么局部计算很重要

如果三个 BE 各自先把数千万条明细按 region 聚合成数百个状态，再通过 Exchange 发送，网络中移动的是小结果；如果先把所有明细集中到一台机器，网络 and 单点内存都会成为瓶颈。

计算靠近数据，先缩小再传输



三个执行维度

机制	作用范围	一句话
MPP	多台 BE	扩展机器与分布式并行
Pipeline	一台 BE 内	用有限线程调度多个可运行任务
向量化	一个算子内	按列式 Block 批量处理多行

4.2 Nereids：从 SQL 到物理计划

Nereids 像数据库编译器：把声明式 SQL 转换为语义等价、预计成本较低的分布式物理计划。它基于 Cascades 思想，同时使用 RBO 与 CBO。

RBO：先消除确定性的浪费

规则	改写前	改写后收益
谓词下推	Join 后才过滤订单	Scan 时过滤，减少 Join 输入
列裁剪	读取整张宽表	只读输出、过滤和 Join 所需列
分区裁剪	调度所有分区	只为命中分区生成 Scan Range
常量折叠	运行时反复计算常量表达式	规划阶段化简
表达式/子查询改写	复杂逻辑结构	转为更适合优化的关系算子

CBO：在多个合理方案中比较成本

- 枚举 Join 顺序、Build Side、Broadcast/Shuffle、聚合阶段和数据分布。
- 估算每个节点的输出行数、行宽、CPU、内存、I/O 与网络成本。
- 选择预计成本最低的计划；‘最低’是基于模型的相对排序，不是运行时间保证。

Memo 与等价计划

Cascades：在等价表达式空间中搜索



物理属性同样参与成本判断。例如某个 Join 的 CPU 很低，但如果它要求先 Shuffle 十亿行，整体仍然可能昂贵。优化器必须同时考虑算子与数据分布。

RBO 与 CBO 对照

维度	RBO	CBO
依据	确定性规则	统计信息与代价模型
典型工作	下推、裁剪、化简	Join 顺序、Join 分布、物理实现
主要风险	规则覆盖不足	统计不准、分布假设不成立
关系	先清理明显浪费	再比较剩余候选计划

调优顺序

不要先写 Hint。先检查统计信息、SQL 写法和数据模型；只有 EXPLAIN 与 Profile 证明优化器选择不合适时，再用 Hint 做受控实验。

4.3 统计信息与基数估算

统计信息是 CBO 的地图。地图错误不会让 SQL 结果错误，却可能让 Join 顺序、Broadcast/Shuffle 和内存判断全部偏离。

两个容易混淆的基数

概念	含义	示例
列基数 NDV	一列有多少个不同值	status NDV=4
算子 Cardinality	某个计划节点预计输出多少行	Filter 预计输出 1000 万行

主要统计指标

指标	作用
row_count / count	表或列的总行数规模
ndv	估算等值过滤、Join 与 Group By
null_count	估算 NULL 条件与有效 Join Key
min / max	估算范围过滤与可能值域
data_size / avg_size_byte	估算 I/O、网络 and 内存

选择率的简化心算

$selectivity \approx 1 / NDV$
 $selectivity \approx \frac{1}{max - min}$
AND
 $Join L \Join R \approx \frac{1}{max(NDV(L.key), NDV(R.key))}$

这些公式用于建立直觉，实际 Nereids 会处理更多边界。它们共同依赖均匀分布、列间独立或值域包含等假设；热点值、列相关性与多对多关系会造成偏差。

误差如何传播

上游估算偏差会放大到所有下游算子



收集与检查

```
ANALYZE TABLE fact_order (order_date, status, customer_id)
WITH SAMPLE PERCENT 10;
```

```
SHOW ANALYZE fact_order;
SHOW TABLE STATS fact_order;
SHOW COLUMN STATS fact_order (order_date, status, customer_id);
```

- 首次历史导入、大规模回灌、数据分布明显变化后，优先更新过滤列、Join列和Group列统计。
- 小表可以全量；超大数据表通常优先采样。统计收集本身也消耗资源。
- 调优时比较 EXPLAIN 估算行数与 Profile 实际行数，寻找第一处巨大偏差。

4.4 PlanFragment、Exchange 与 MPP

MPP 让一条 SQL 同时使用多台 BE；Fragment 是计划模板，Instance 是它在具体 BE 上的运行副本，Exchange 负责在副本之间改变数据分布。

层级关系

从查询到 BE 内部任务



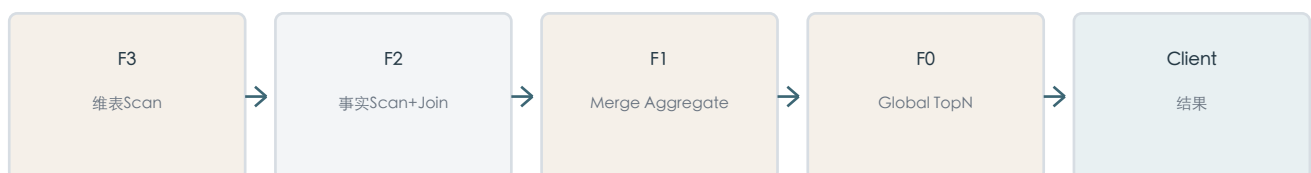
为什么在 Exchange 处切 Fragment

当下游要求新的数据分布时，上游通过 `DataStreamSink` 发送列式 Block，下游通过 `ExchangeNode` 接收。发送端与接收端位于不同 Fragment，形成分布式数据流边界。

交换方式	语义	常见用途	主要风险
Gather / Unpartitioned	汇聚到少数或一个接收端	最终结果、全局TopN	过早单点化
Hash Partition	相同Key到同一接收端	Join、Group By	热点Key倾斜
Broadcast	完整复制到所有接收端	小表Join大表	网络和Hash表按实例放大
Random	均衡分发但不保Key属性	调整并行度	不能直接满足按Key计算

一条聚合查询的 Fragment DAG

Fragment 之间由 Broadcast、Hash Exchange 与 Gather 连接



为什么 MPP 快但不线性

- 增加 BE 会增加 CPU、内存、扫描与网络能力。
- 但调度、Exchange、最终合并、串行阶段与倾斜不会同比消失。
- 阶段时间由最慢 Instance 决定；一台慢节点或一个热点Key就能拖住全局。

Doris 3.0 两种部署形态

模式	Scan来源	MPP不变的部分
存算一体	BE优先读取本地Tablet副本	Fragment、Instance、Exchange、Pipeline
存算分离	BE读取共享存储并利用本地缓存	仍由FE规划并在计算BE间交换中间数据

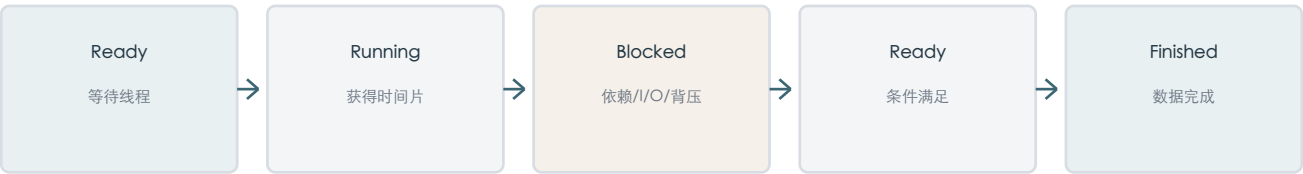
4.5 Pipeline 与向量化

Pipeline 决定哪个 Task 在什么时候占用 CPU；向量化决定 Task 拿到数据后怎样按 Block 批量计算。两者属于不同层次。

为什么不让任务永久绑定线程

高并发下，一任务一线程容易造成线程膨胀、上下文切换和大量阻塞线程。PipelineTask 在等待网络、I/O、下游缓冲或算子依赖时可以让出 Worker，线程转而执行其他 Ready Task。

PipelineTask 状态循环



阻塞算子如何切 Pipeline

算子	为什么形成边界	典型依赖
Hash Join	Build先构建Hash表，Probe后使用	Probe等待Build Ready
Aggregate	输入阶段更新Hash表，输出阶段遍历状态	Source等待Sink完成
Sort	通常先收集并排序，再输出有序结果	Sort Source等待排序完成
Exchange	接收端等待远端数据到达	网络与缓冲依赖

背压与 Local Exchange

- 下游缓冲区满时，上游暂停并让出线程，避免中间数据无限堆积。
- Local Exchange 在同一 BE 内重分发 Block，用于平衡 Task、按Key分区或调整上下游并行度。
- 跨 BE Exchange 有网络成本；Local Exchange 主要是本地队列、内存与同步成本。

向量化为什么快

逐行执行	向量化 Block
每行每算子多次调用	每批每算子调用一次，摊薄调度开销
行内多类型字段交错	同类型列连续排列，Cache友好
大量分支与间接跳转	批量选择向量与紧凑循环
难以使用SIMD	数值比较、过滤、聚合可批量执行

磁盘列式布局与内存向量化自然衔接



不要盲调 batch_size

Block太小会增加函数调用和调度；太大则增加内存、Cache压力和小查询尾延迟。3.0.8已经以向量化为正常执行路径，通常无需‘开启’，也不应在没有Profile证据时修改批次大小。

4.6 Scan 与多层数据裁剪

最快的数据处理，是根本不读取。Scan优化的主线不是把全表读得更快，而是让绝大多数数据在进入执行引擎前就被排除。

从粗到细的裁剪漏斗



分区裁剪与 Tablet 裁剪

维度	分区裁剪	Tablet裁剪
依据	PARTITION BY Range/List	DISTRIBUTED BY HASH
典型条件	日期范围、列表值	分桶列等值或可确定IN
EXPLAIN	partitions=N/M	tablets=N/M
效果	跳过整个业务范围	在命中分区内定位Bucket

让条件保持可推导

```
--
WHERE order_date >= '2026-08-01'
    AND order_date < '2026-09-01'

--
WHERE DATE_FORMAT(order_date, '%Y-%m') = '2026-08'
```

- 尽量保持列为原始形式，让常量与列类型匹配。
- OR连接一个不受分区约束的条件时，任意分区都可能命中。
- 复合分桶Key需要足够条件才能确定Hash目标。

列裁剪与 SELECT *

过滤列、Join列和输出列必须读取，其他列可以裁掉。宽表中的大字符串列即使行数不多也会制造大量I/O、解压与Block内存；SELECT * 会迫使引擎保留所有输出列。

ZoneMap 的确定性跳过

```
Page-1: min(amount)=10, max(amount)=80
Query : amount > 100
      : Page-1
```

ZoneMap能证明‘肯定没有’时跳过；范围相交只表示‘可能有’，仍需读取并执行向量化谓词。BloomFilter、倒排索引和Runtime Filter可以继续缩小候选。

EXPLAIN 扫描检查

```
V0lapScanNode
  PREDICATES: order_date >= ... , customer_id = 10086
  partitions=1/12
  tablets=1/32
  cardinality=...
```

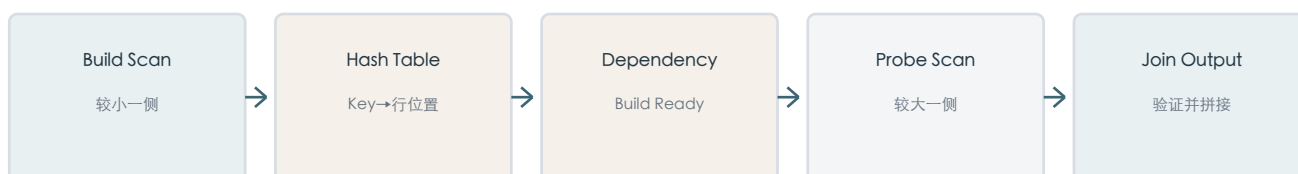
先减少扫描范围，再优化计算方式，最后才增加资源。

4.7 Hash Join 与四种分布策略

Join正确执行的物理条件是：相同 Join Key 的两侧数据必须在同一个 Join Instance 相遇。四种策略的差别，本质上是谁移动、移动多少。

Build 与 Probe

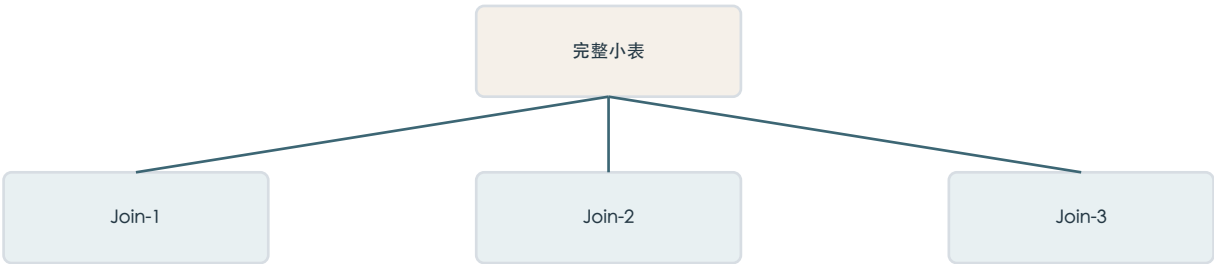
Hash Join：先构建，再流式探测



Build Side的内存不只是原始列，还包括Hash桶、Key、行引用与对齐开销。‘小表’应按过滤、投影、聚合后的真实行数与行宽判断，而不是原表名称或总行数。

Broadcast

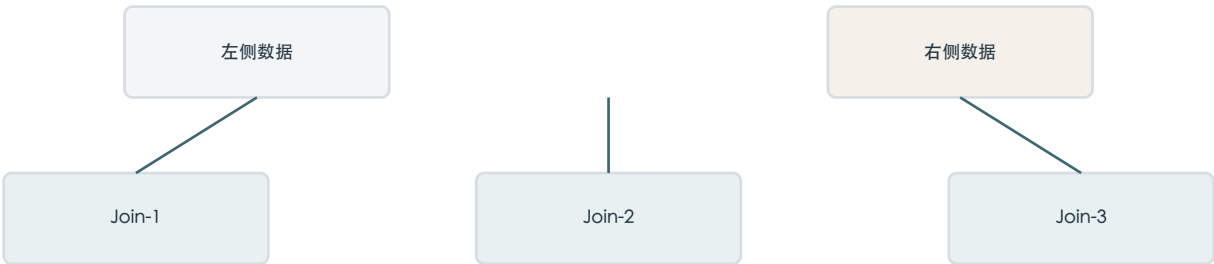
Broadcast：小侧完整复制到所有 Join Instance



- 近似网络量为 $N \times R$ ；每个 Join Instance 各保存一份 Hash 表。
- 适合大事实表 Join 过滤后的小维表；统计错误会把错误 Broadcast 放大成网络 and 内存事故。

Partition Shuffle

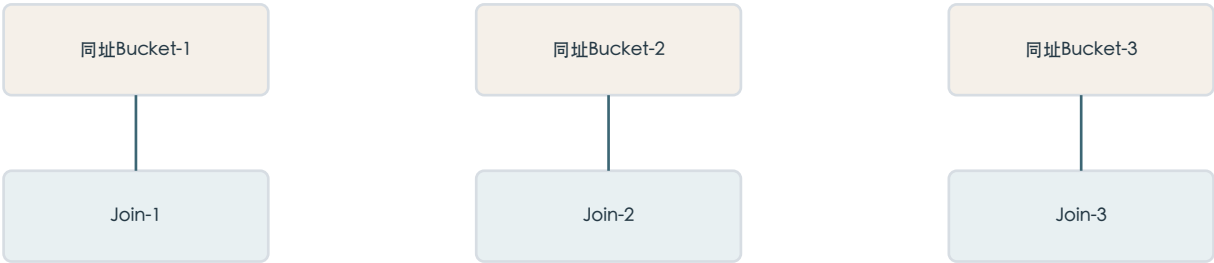
Shuffle：两侧按 Join Key 使用相同 Hash 规则重分发



- 近似网络量为 $L + R$ ；适合两侧都较大且当前分布不兼容。
- 适用范围广，但热点Key会集中到同一个接收端。

Bucket Shuffle 与 Colocate

复用已有Bucket：少移动一侧，或两侧都不移动



策略	近似Join网络量	要求
Bucket Shuffle	只移动需要重分发的一侧 R	Join Key与可复用Bucket分布兼容
Colocate	0	同组、同Bucket规则、对应副本同址且Group稳定

四种策略总表

策略	网络量	灵活性	主要风险
Broadcast	$N \times R$	高	右侧变大后复制爆炸
Shuffle	$L + R$	高	大网络与Key倾斜
Bucket Shuffle	R	中	布局 and 分区条件不满足
Colocate	0	低	布局约束、Group不稳定、并行度不足

Join顺序与结果膨胀

- 先让选择性强的小维表过滤客户，再关联事实表，通常比先产生巨大中间结果更合理。
- 同一个Key在左右两侧各有1000行时，可产生100万行多对多结果；这不是普通倾斜，而是基数膨胀。
- Outer Join受NULL补齐语义限制，不能像Inner Join一样任意重排或过滤保留侧。

Hint 只做最后验证

```
SELECT COUNT(*)
FROM fact_order o
JOIN [broadcast] dim_customer c
  ON o.customer_id = c.customer_id;
```

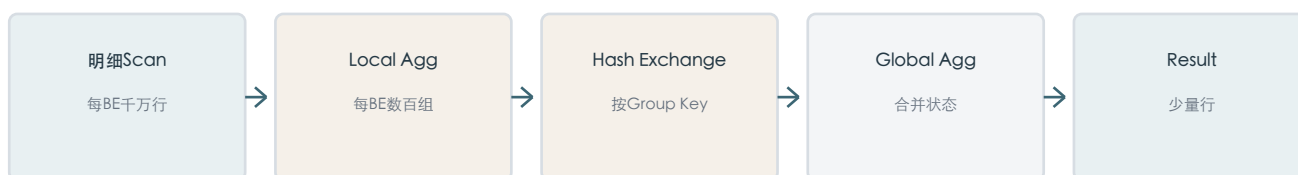
Hint可能在当前数据规模有效，却在数据增长后成为固定坏计划。使用前后都应保存EXPLAIN与Profile。

4.8 聚合、Sort、TopN 与 Runtime Filter

聚合和TopN的关键不是把所有数据算完再缩小，而是尽早维护可合并状态与动态阈值；Runtime Filter则把执行期已知信息反推回Scan。

局部聚合与全局聚合

Exchange 前聚合，网络只传中间状态



SUM只需合并局部和；AVG需要保存Sum+Count；HLL与Bitmap也有可合并状态。Group Key NDV决定Hash表与缩减率：1亿行变600组收益巨大，1亿行变9000万组则几乎没有缩减。

Sort 与 TopN

维度	全量Sort	TopN
目标	所有行形成全局顺序	只保留前N名
近似复杂度	$O(M \log M)$	$O(M \log N)$
中间内存	随M增长	主要随N增长
分布式	局部排序后全局Merge	Local TopN候选后Global TopN

- 大OFFSET仍要找到OFFSET+LIMIT个候选；连续翻页更适合基于排序Key的游标。
- 当排序列匹配表Key前缀时，可利用已有顺序减少读取。
- 两阶段延迟物化先读排序列找行号，再回读TopN完整列，宽表收益明显。

Join Runtime Filter

运行时将 Build Side 信息推回 Probe Scan



类型	性质	适合场景
IN	精确Key集合	Build不同Key较少
Bloom	无假阴性、允许假阳性	Key集合较大，先排除肯定不存在
Min/Max	值域包络，成本低	数据范围集中或列有序
IN_OR_BLOOM	按实际Key数量自适应	执行期选择结构

Runtime Filter 的边界

- 它只能减少计算，最终Join仍需验证，不能改变SQL结果。
- Scan立即开始与等待Filter存在时机矛盾；到达太晚会降低过滤收益。
- Left Outer Join不能随意用右侧Key过滤左侧保留行。

TopN Runtime Filter

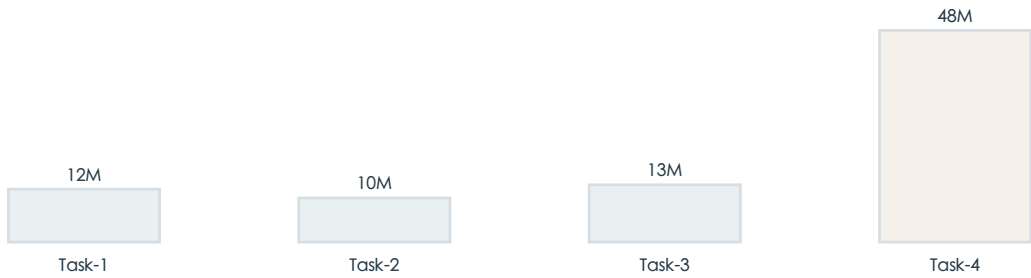
TopN维护当前第N名阈值：升序时大于当前边界的数据不再有资格，降序时小于边界的数据可被排除。阈值随更优Block到来不断收紧，并可结合ZoneMap跳过Page。

```
EXPLAIN TOPN OPT / SORT LIMIT / OPT TWO PHASE
Profile RuntimePredicateSegment
```

4.9 数据倾斜、并行度与内存

并行度的目标不是最大，而是让瓶颈阶段充分使用CPU且不过度复制状态。数据倾斜时，平均值会掩盖真正的单点。

平均值看似正常，最慢 Task 决定整个阶段耗时



提高并行度的收益与代价

可能收益	同步代价
更多核心并行处理Scan/Join/Agg	更多Task、Local Exchange和同步
单个Task处理更少数据	更多缓冲区、状态分片与内存
重查询单次延迟下降	高并发吞吐可能下降

先判断瓶颈类型

症状	更可能的瓶颈	并行度是否优先
CPU未满、算子ExecTime高、行数均匀	CPU计算	可小步提高并验证
Scan等待高、磁盘带宽满	I/O	通常不是
Exchange远程字节大、网络满	网络	通常不是
WaitForDependency高	上游依赖	先找被等待者
max远大于avg	数据/资源倾斜	增加Task通常治标无效

五类倾斜

- Tablet倾斜：分桶Key或Bucket设计导致存储量不均。
- Join Key倾斜：热点Key在Hash Shuffle后进入同一Instance。
- Group By倾斜：热点Group集中到一个聚合Task。
- Join膨胀：热点Key两侧重复造成多对多输出。
- 节点资源倾斜：数据相同但某BE磁盘、网络或负载异常。

Profile识别

ProbeRows: avg=165K, max=219M, min=1,984
ExecTime : avg=166ms, max=10.9s, min=8ms
 : Task

查询内存构成

算子	主要内存结构
Hash Join	Build Hash表、Key与行引用
Aggregate	Group Hash表、聚合状态、String Arena
Sort/TopN	排序Key、候选行与中间归并
Exchange	发送/接收缓冲与列式Block
Pipeline	每Task LocalState、队列和状态分片

内存超限排查

```
SHOW VARIABLES LIKE 'exec_mem_limit';
--
SET exec_mem_limit = 8G;
```

在3.0系列中常见默认值为2 GB，但集群可能已修改。遇到MEM_LIMIT_EXCEEDED应先定位Join Build、Group NDV、Sort、广播复制和并发，而不是无限提高限制。Spill用磁盘换内存，是稳定性保护，不是加速。

Workload Group

单SQL优化解决效率，多查询治理需要资源隔离。Workload Group可对CPU、内存、最大并发和队列进行约束，避免离线重查询拖垮实时看板。

4.10 EXPLAIN 与 Profile 慢查询诊断

EXPLAIN 是出发前的导航路线，Profile 是执行后的真实轨迹。调优要比较两者，并沿数据流寻找第一处估算或规模开始异常的节点。

工具分工

工具	是否执行SQL	主要回答
EXPLAIN	否	Fragment、Scan、Join、Exchange、聚合与估算
EXPLAIN VERBOSE	否	更详细的表达式、Slot与输出信息
EXPLAIN SHAPE PLAN	否	紧凑查看Nereids物理形状、Join顺序和分布
EXPLAIN PHYSICAL PLAN	否	物理算子和分布属性
Query Profile	是	实际行数、耗时、内存、等待与倾斜

生成 Profile

```
SET enable_profile = true;
SHOW VARIABLES LIKE 'enable_profile';

-- SQL
SHOW QUERY PROFILE;
```

建议在诊断Session开启。长期为所有高频小查询收集详细Profile会增加FE采集、保存与查询负担。

MergedProfile 与明细 Profile

层次	用途
MergedProfile	汇总多个BE/Task的sum、avg、max、min，先找热点和峰值
Execution Profile	下钻到具体BE、Instance、PipelineTask，确定异常位置

通用指标

指标	含义	阅读提醒
ExecTime	算子自身执行时间	并行Task时间不能直接相加等于墙钟时间
RowsProduced	Source/中间算子输出行数	寻找数据首次膨胀或缩减失败
InputRows	Sink接收行数	与上游输出交叉确认
MemoryUsagePeak	算子峰值内存	Hash表、Arena等结构需单独看
WaitForDependency	等待依赖时间	等待者常是受害者，要追被等待的上游
max/avg/min	不同Task的差异	max远高于avg通常指向倾斜

从 Scan 向上读

不从最上面的 Result 开始猜，沿真实数据流逐级核对



标准诊断流程

1. 记录SQL、Query ID、耗时、数据规模和Session变量。
2. EXPLAIN检查分区/Tablet裁剪、Join顺序与分布、聚合阶段、TopN和Runtime Filter。
3. 执行并获取Profile，对比估算与实际行数。
4. 用max/avg/min识别数据或节点倾斜。
5. 定位第一处行数、字节、时间或内存开始异常的节点。
6. 一次只修改一个因素，重新EXPLAIN与Profile。
7. 同时验证结果正确、延迟、CPU、网络、内存与并发稳定性。

教学案例：三层根因

```
1: 8 partitions=12/12
  : DATE_FORMAT(order_date, ...)
  : >= 81 AND < 91

2: 103000Broadcast
  :
  : ANALYZECBO

3: ProbeRows maxavg
  : customer_idKey
  : /
```

为什么找第一处异常

最终Sort慢30秒，可能只是因为上游错误Join产生了20亿行。如果只给Sort加内存，只是在帮助它更快处理不该出现的数据。沿数据流找到第一次偏离合理规模的位置，才能治本。

第四章复习题

1. 为什么说 SQL 只描述‘要什么’，而物理执行计划决定‘怎么做’？
2. 逻辑计划、物理计划、PlanFragment 和 Fragment Instance 有什么区别？
3. RBO 与 CBO 各自解决什么问题，为什么二者不能互相替代？
4. 为什么统计信息过期可能把一个合理的 Broadcast Join 变成内存事故？
5. 列 NDV 与算子 Cardinality 有什么区别？多个估算误差为什么会逐层放大？
6. PlanFragment 为什么常在 Exchange 附近切分？Gather、Hash、Broadcast 各自改变什么？
7. MPP、Pipeline 与向量化分别解决哪个层次的问题？
8. PipelineTask 阻塞时为什么应该让出 Worker？背压又解决什么问题？
9. 分区裁剪、Tablet 裁剪、列裁剪和 ZoneMap 的作用层级有什么不同？
10. 为什么在分区列上包裹函数、使用 SELECT * 会削弱扫描效率？
11. Hash Join 的 Build 与 Probe 如何协作？为什么要让较小一侧构建 Hash 表？
12. Broadcast、Shuffle、Bucket Shuffle 和 Colocate 的网络成本与约束分别是什么？
13. Join Key 倾斜与 Join 结果膨胀有什么区别？Profile中如何辨认？
14. 为什么局部聚合应该尽量发生在 Exchange 之前？Group Key 高基数有什么影响？
15. TopN 为什么比全量 Sort 便宜？大 OFFSET 为什么仍可能很慢？
16. Join Runtime Filter 的 IN、Bloom、Min/Max 各有什么性质与边界？
17. 为什么提高 parallel_pipeline_task_num 可能降低系统吞吐并增加内存？
18. 遇到 MEM_LIMIT_EXCEEDED 时，为什么不应第一时间无限提高 exec_mem_limit？
19. EXPLAIN 与 Query Profile 分别回答什么问题？为什么必须配合？
20. 怎样沿数据流定位慢查询的‘第一处异常节点’？请给出完整排查顺序。

自测方法

每题先用一句话给结论，再补齐对象关系、数据流和一个可观测证据。答案请使用独立的 notes-04-a，避免边看边背。

术语速查

术语	一句话含义
Cardinality	计划节点预计或实际输出的行数
NDV	某列不同值的数量
Fragment	FE下发到BE的一段分布式物理计划模板
Instance	Fragment在某台BE上的运行副本
Exchange	Fragment之间改变数据位置或分布的边界
Pipeline	由Source、中间Operator和Sink组成的可调度执行链
Block	算子之间传递的一批列式数据
Build / Probe	Hash Join的Hash表构建侧与查找侧
Runtime Filter	执行期生成并下推的辅助过滤条件
Straggler	拖慢整个并行阶段的最慢任务

参考资料

以下链接用于核对 Doris 3.0.8 相关机制；3.0 页面优先，3.x/dev 页面用于补充跨版本稳定的底层概念。

- [1] [Doris 3.0 Query Optimizer](#)
- [2] [Doris 3.0 Statistics](#)
- [3] [Doris 3.0 ANALYZE](#)
- [4] [Doris 3.0 Pipeline Execution](#)
- [5] [Vectorized Execution](#)
- [6] [MPP Architecture](#)
- [7] [Partition Pruning](#)
- [8] [Doris 3.0 Runtime Filter](#)
- [9] [Doris 3.0 TopN Optimization](#)
- [10] [Doris 3.0 Data Skew Handling](#)
- [11] [Parallelism Tuning](#)
- [12] [Query Profile](#)
- [13] [Doris 3.0 Query Memory Error](#)
- [14] [Release 3.0.8](#)